

Report DNA Sequence Alignment

Studente: Mattia Prisco 2046234

Corso: Programmazione Multicore

Implementazioni

Introduzione

Il progetto mira a parallelizzare un settore specifico dell'algoritmo basato su forza bruta di allineamento delle sequenze di DNA, ed in esso è stato trovato come fonte di maggiore tempo computazionale un ciclo for, dove per ogni pattern viene individuata nella sequenza iniziale una sottosequenza uguale al pattern stesso. Inoltre, una volta riscontrato un risultato, l'iterazione si conclude andando a raccogliere dati relativi al riscontro, utili per i successivi calcoli di checksum dell'algoritmo. Risulta quindi chiaro che, all'aumentare dei pattern, della loro lunghezza e della lunghezza della sequenza iniziale, aumenti il tempo di esecuzione del programma. Le implementazioni delle versioni che vengono proposte (OMP+MPI e CUDA) mirano in particolare all'equidistribuzione dei pattern. Prima di procedere ad esaminare ognuna di esse, è importante sottolineare che questa non è stata la prima soluzione ipotizzata: inizialmente, si è provato a procedere distribuendo la lunghezza della sequenza sulle varie risorse disponibili, concedendo un offset agli estremi di ogni porzione. Tuttavia, all'aumentare del numero di pattern questa soluzione non sempre risulta efficace, soprattutto se la lunghezza della sequenza non è molto grande.

OMP+MPI

Nella versione combinata in OMP+MPI, come spiegato precedentemente, l'idea generale sviluppata è di distribuire equamente, tra i processi, i pattern e di parallelizzare il ciclo for di elaborazione e ricerca dei pattern attraverso i threads generati da ogni processo. Per la suddivisione dei pattern, è stato definito per ogni rank un limite iniziale e finale, in base a quanti processi sono richiesti a tempo di esecuzione.

```
// Distribute the patterns among the processes
int parziale = pat_number / size;
int resto = pat_number % size;
int inizio = rank * parziale + resto;
int fine = (rank+1) * parziale + resto;
```

Dopo di ch , una volta ottimizzato, si   passati alla parallelizzazione del for principale: per fare ci ,   stata usata la clausola **omp parallel for**. In essa, parametri come **private** non sono stati necessari, in quanto variabili private sono gestite internamente al ciclo. Tuttavia   stato fondamentale l'aggiunta dei parametri **reduction(+:pat_matches)** e **reduction(+:seq_matches[seq_length])**, poich  risulterebbe problematico il salvataggio dei risultati ogniqualvolta viene trovato un pattern nella sequenza (le risorse soggette a modifica sono 'pat_matches', 'pat_found' e 'seq_matches'). Facendo in tal modo, ogni thread crea una propria variabile locale, e alla fine di tutta l'esecuzione dei threads, avviene la reduction che permette ad ogni processo di avere tutti i risultati. Infine,   stato adottato un modello di scheduling di tipo guided, attraverso la clausola **schedule(guided)**,

che permette di distribuire il carico di pattern per thread dinamicamente diminuendo man mano i chunk. Questo è stato fatto in quanto è imprevedibile il tempo con cui ogni thread elabora il blocco di pattern, e quindi migliorerebbe le prestazioni di uno scheduling di tipo static, ma non si vuole nemmeno aumentare troppo l'overhead che invece si otterrebbe con uno scheduling di tipo dynamic. È quindi una via di mezzo efficace per un uso efficiente delle risorse.

```
// used omp parallelism
#pragma omp parallel for reduction(+:pat_matches) reduction(+:seq_matches[:seq_length]) schedule(guided)
for( int pat=inizio; pat < fine; pat++ ) {
    unsigned long lunghezza_path = pat_length[pat];
    char *current_pattern = pattern[pat];
    unsigned long max_length = seq_length - lunghezza_path;
    /* 5.1. For each possible starting position */
    for( unsigned long start=0; start <= max_length; start++ ) {
        unsigned long lind = 0;
        if (sequence[start] != current_pattern[0]) continue;
        /* 5.1.1. For each pattern element */
        while( lind<lunghezza_path && sequence[start + lind] == current_pattern[lind] ) {
            lind++;
        }
        /* 5.1.2. Check if the loop ended with a match */
        if ( lind == lunghezza_path ) {
            pat_matches++;
            pat_found[pat] = start;
            for( unsigned long ind=0; ind<lunghezza_path; ind++ ) {
                seq_matches[ start + ind ] ++;
            }
            break;
        }
    }
}
```

Per il completamento del programma, è stato necessario raggruppare i dati, e perciò state usate le funzioni **MPI_Gather** e **MPI_Reduce**; la prima per raccogliere tutti i 'pat_found' parziali (in ogni singolo rank) in un unico vettore, mentre la seconda per fare la somma dei risultati ottenuti in 'pat_matches' (somma di interi) e 'seq_matches' (somma di vettori). Tuttavia, si è considerato il caso in cui il numero di pattern in input non è esattamente un multiplo del numero di processi, quindi il compito assegnato a MPI_Gather è quello di raccogliere tutti i risultati dei 'pat_found' parziali, tranne il resto non equidistribuito e gestito dal rank 0. L'operazione finale di collecting avviene quindi in un apposito ciclo for, molto rapido nella sua esecuzione e che quindi non necessita di parallelizzazione. Importante sottolineare che non sono stati considerati soluzioni che usassero MPI_Barrier o sistemi di send e receive tra processi (per fare in modo che il tempo computazionale fosse mantenuto moderato) e che è bastato la clausola MPI_Init() ad inizio programma in quanto non è stato necessario che threads comunicassero tra loro con funzioni MPI.

```
// MPI Gather and Reduce, to collect results in the root process
MPI_Gather(&pat_found[inizio], parziale, MPI_UNSIGNED_LONG, pat_foundRoot, parziale, MPI_UNSIGNED_LONG, 0, MPI_COMM_WORLD);
if (rank==0){
    // Collect the remaining patterns in the root process, following their order
    for (int i = 0; i < pat_number; i++) {
        // if the pattern is before the remainder, copy from pat_foundRoot
        if (i < parziale) {
            pat_found_res[i] = pat_foundRoot[i];
        }
        // if the pattern is in the remainder, copy from pat_found
        else if (i < parziale + resto) {
            pat_found_res[i] = pat_found[i];
        }
        // if the pattern is after the remainder, copy from pat_foundRoot
        else {
            pat_found_res[i] = pat_foundRoot[i - resto];
        }
    }
}

MPI_Reduce(&pat_matches, &pat_matches_root, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(seq_matches, seq_matchesRoot, seq_length, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

CUDA

Nella versione CUDA, l'idea generale sviluppata è di parallelizzare il ciclo for attraverso un cuda kernel in modo da far svolgere ai threads dei blocchi delle GPU un'elaborazione dati veloce e efficiente. Per fare ciò, si è dapprima copiato le risorse utili per il **cuda kernel** nelle memorie delle GPU, e successivamente si è calcolato il numero e la dimensione dei blocchi usati. Da notare che in questa implementazione non è stato elaborato un sistema che tenta di utilizzare **shared memory** o **constant memory** per salvare in ogni blocco della GPU sia la sequenza sia i pattern, in quanto la quantità di pattern e la lunghezza della stringa poste in input potrebbero richiedere più memoria shared del consentito. Al contrario, è stato molto utile l'uso della **pinned memory** per velocizzare lo scambio di dati tra CPU e GPU, che ha migliorato il tempo di esecuzione dell'8% circa.

```
char *d_sequence;
CUDA_CHECK_FUNCTION( cudaHostRegister( sequence, sizeof(char) * seq_length, cudaHostRegisterDefault ) );
CUDA_CHECK_FUNCTION( cudaMalloc( &d_sequence, sizeof(char) * seq_length ) );
CUDA_CHECK_FUNCTION( cudaMemcpy( d_sequence, sequence, sizeof(char) * seq_length, cudaMemcpyHostToDevice ) );
CUDA_CHECK_FUNCTION( cudaHostUnregister( sequence ) );
```

Si è quindi deciso di mantenere un kernel con risorse direttamente nella **global memory**, tranne due variabili intere (la lunghezza della sequenza e il numero di pattern) che vengono posti direttamente nei registri dei cuda threads per una maggiore efficienza. È stato posto l'accento quindi sull'ottimizzazione dell'algoritmo ove fosse possibile, e sull'uso della clausola **atomicAdd**, in modo che sezioni critiche non venissero corrotte erroneamente.

```
__global__ void pattern_search_kernel(const char* d_sequence, int* d_pat_matches, unsigned long* d_pat_found, int* d_seq_matches,
unsigned long* d_pat_lengths, const char ** d_patterns, unsigned long seq_length, int pat_number) {
    // Each thread will handle one pattern
    int threadId = threadIdx.x + blockIdx.x * blockDim.x;
    int pat = threadId;
    unsigned long lind;

    // Check if the thread is within the range of patterns
    if (pat >= pat_number) return;

    const char* pattern = d_patterns[pat];
    unsigned long pat_len = d_pat_lengths[pat];
    unsigned long max_len = seq_length - pat_len;

    for (unsigned long start = 0; start <= max_len; start++) {
        unsigned long i = 0;
        if (d_sequence[start] != pattern[0]) continue;
        unsigned long current_index = start;
        while (i < pat_len && d_sequence[current_index] == pattern[i]) {
            i++;
            current_index++;
        }
        if (i == pat_len) {
            atomicAdd(d_pat_matches, 1);
            d_pat_found[pat] = start;
            current_index = start;
            for (unsigned long k = 0; k < pat_len; ++k) {
                atomicAdd(&d_seq_matches[current_index], 1);
                current_index++;
            }
            break;
        }
    }
}
```

Testing

Per la misurazione e l'analisi dei tempi di esecuzione, sono state applicate 50 esecuzioni per ogni combinazione di risorse e test. Per la versione OMP+MPI sono stati combinati threads e processi per un totale di 32 flussi di esecuzione paralleli, mentre per la versione CUDA è stato usato un kernel definito da blocchi di dimensione 256, 512 e 1024 threads. I test usati sono:

40000 0.35 0.2 0.25 50000 1250 1250 50000 1250 1250 50 100 M 4353435

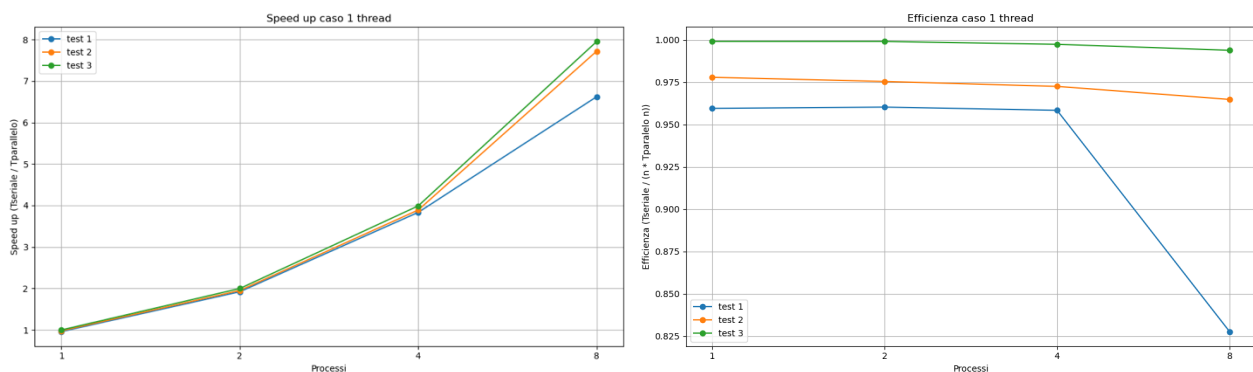
83500 0.35 0.2 0.25 90000 7500 2500 90000 7500 2500 50 100 M 4353435

167000 0.35 0.2 0.25 90000 7500 2500 90000 7500 2500 50 100 M 4353435

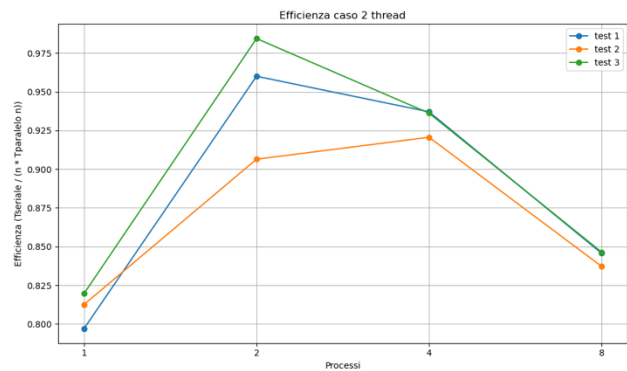
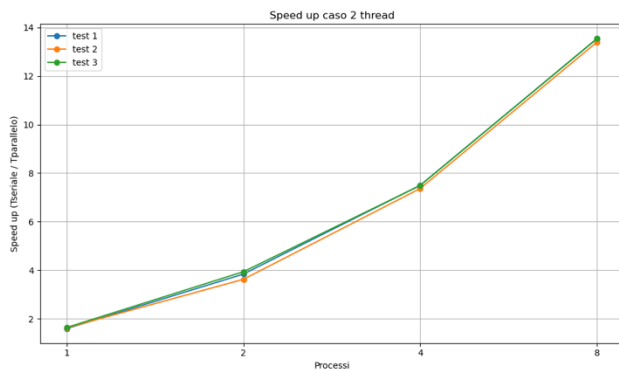
MPI+OMP

Per l'analisi dei dati nella versione ibrida MPI+OMP sono stati condotti 36 test in totale, variando il numero di processi da 1 a 8 e di thread da 1 a 4. L'utilizzo di risorse più elevate non è stato possibile a causa delle forti limitazioni di accesso imposte dal cluster.

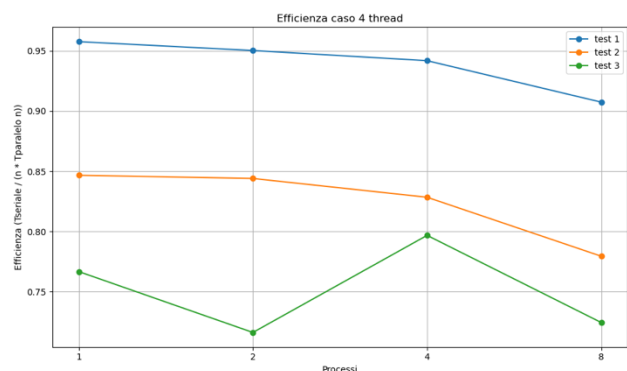
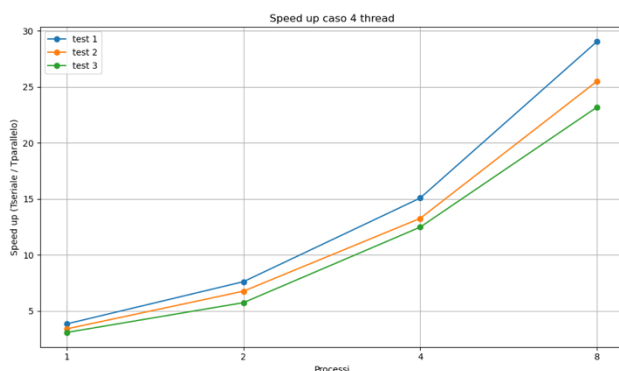
L'analisi si è concentrata d'apprima su due aspetti: lo speedup e l'efficienza ottenuti in ciascun test. Per quanto riguarda invece la dispersione delle misurazioni temporali, si è fatto riferimento al caso specifico con 4 thread, ritenuto rappresentativo per questo tipo di valutazione.



Nel primo caso, cioè usando un solo thread per processo, si può evincere come il solo parallelismo con MPI permette di mantenere una buonissima efficienza e uno speed up crescente. Questo conferma quanto detto nella descrizione della soluzione discussa precedente, cioè che si è riusciti a mantenere un parallelismo efficiente grazie alla mancanza di funzioni bloccanti. Si registra comunque un leggero aumento dello speed up e un'efficienza più costante all'aumentare della grandezza dell'input e dei processi, probabilmente perché l'overhead viene compensato da lavoro utile da parte di ogni processo.



Nel secondo caso, ove disponiamo di 2 thread per processo, si osserva uno speed up ancora alto, con una concavità leggermente meno accentuata rispetto al caso precedente, ma stavolta condivisa da entrambi i test, mentre un'efficienza che, probabilmente per un overhead dovuto alla gestione delle risorse, ha valori più bassi al minimo e al massimo del numero di processi. Ciò si nota in tutti i test, con il test alto che una quantità media di risorse ha l'efficienza più alta verificata durante tutta la misurazione. Tuttavia, l'efficienza rimane abbastanza vicina ai valori ideali, con un minimo totale poco inferiore al 0.8.



Infine, nell'ultimo caso, disponiamo del maggior numero di risorse: da 8 fino a 32 flussi di esecuzione parallela. Possiamo osservare che lo speedup tende ad assumere un andamento ancora esponenziale, ma stavolta la concavità più accentuata la ha il test più piccolo. L'efficienza si presenta per la prima volta decrescente a partire da 1 processi e 4 thread e, rispetto agli altri test, il grafico mostra prestazioni migliori sul caso con input più piccolo, fenomeno che può essere spiegato con la presenza di oversubscription sui core del cluster, che ha comportato un maggiore presenza di outliner nei test con input più grande. Si mantengono comunque buone prestazioni.

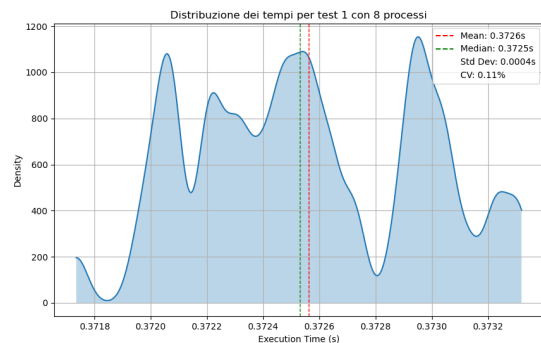
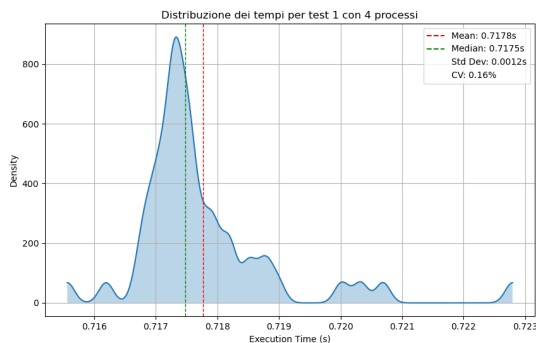
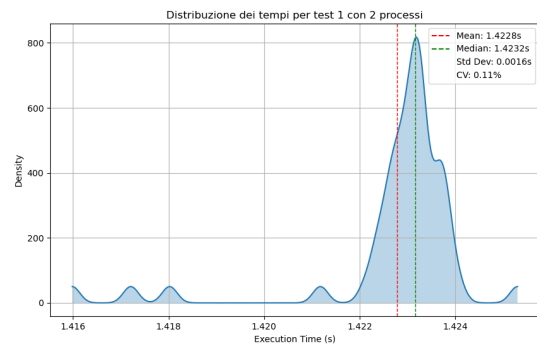
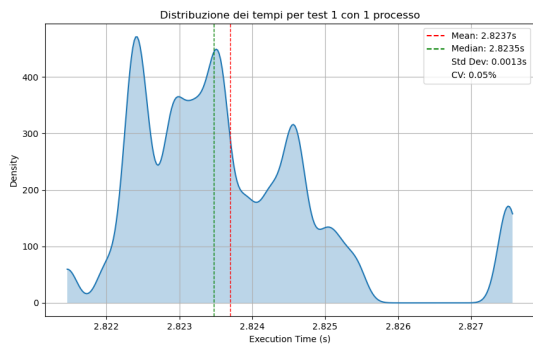
Una considerazione finale riguarda l'impatto della combinazione tra numero di processi e thread: si sono osservate differenze di prestazioni significative all'aumentare dei processi e alla diminuzione dei thread, soprattutto con input molto alti. La seguente tabella rappresenta chiaramente questo comportamento.

efficienza ($T_{\text{seriale}} / (n * T_{\text{parallelo}})$)			
	test 1	test 2	test 3
1 thread e 8 processi	0,827572	0,964820	0,993845
2 thread e 4 processi	0,937088	0,920621	0,936205
4 thread e 2 processi	0,950290	0,844064	0,716189

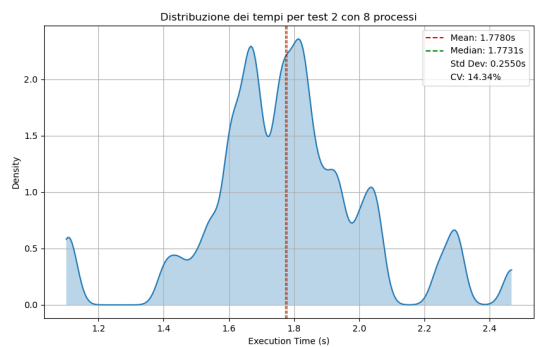
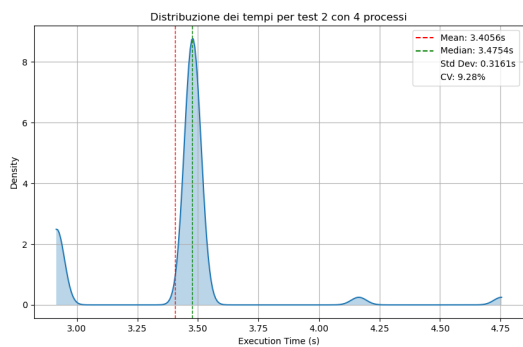
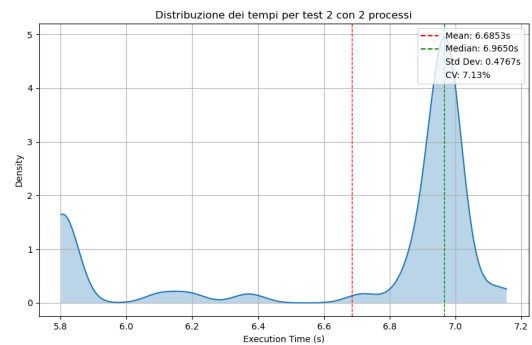
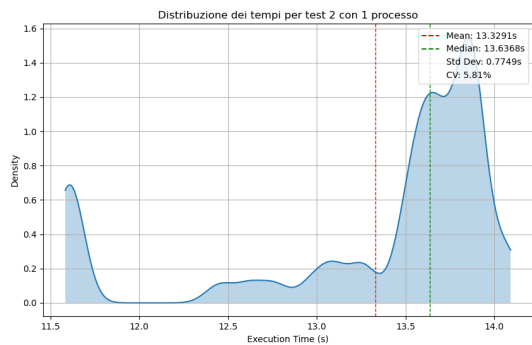
Questo comportamento può essere spiegato dal fatto che, all'aumentare del numero di thread, si introduce un overhead crescente legato alla costruzione delle strutture dati utilizzate nella parallelizzazione del ciclo principale, nonché alle operazioni di reduction. In particolare, le reduction in OMP diventano meno efficienti con un numero elevato di thread, poiché richiedono una sincronizzazione tra i thread per combinare i risultati parziali, il che può rallentare l'esecuzione complessiva.

Tutte le osservazioni descritte trovano conferma nei grafici riportati in seguito, che mostrano le distribuzioni dei tempi di esecuzione per i tre test, nel caso di 4 thread e un numero di processi variabile da 1 a 8. Si evince infatti che all'aumentare delle risorse, andando anche oltre i limiti hardware, e dei test, si perde consistenza e stabilità dei dati.

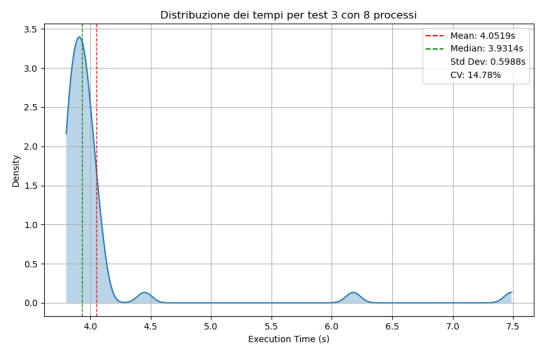
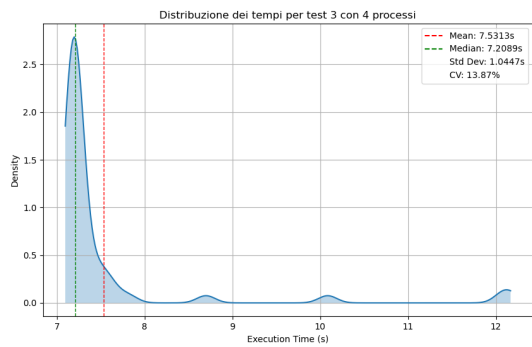
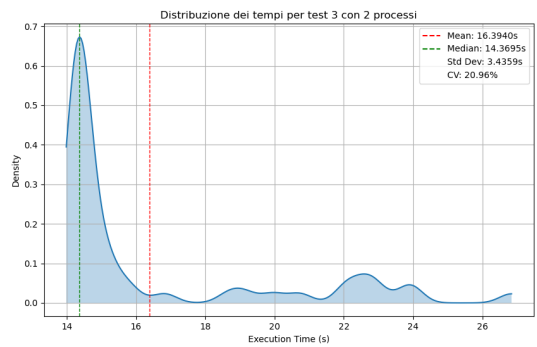
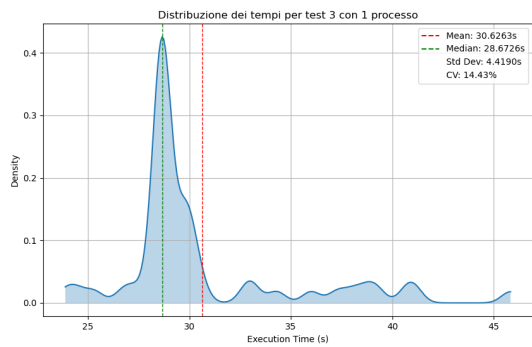
Primo test



Secondo test



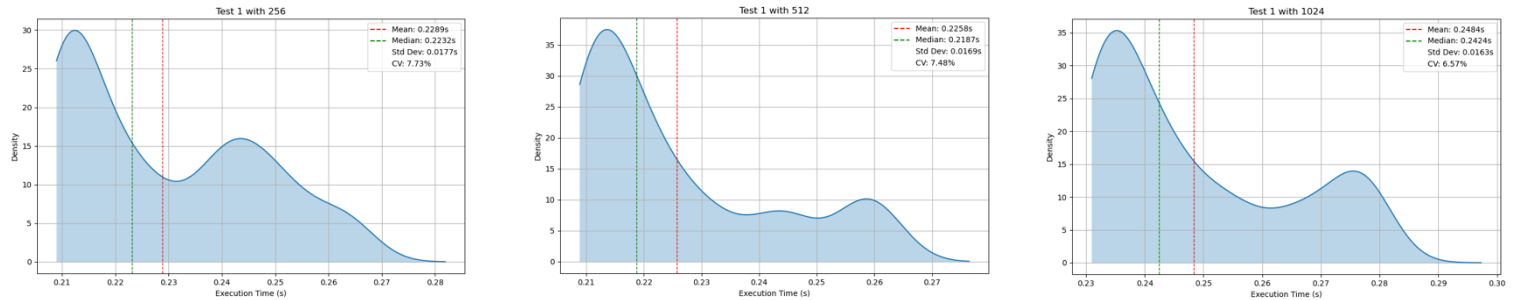
Terzo test



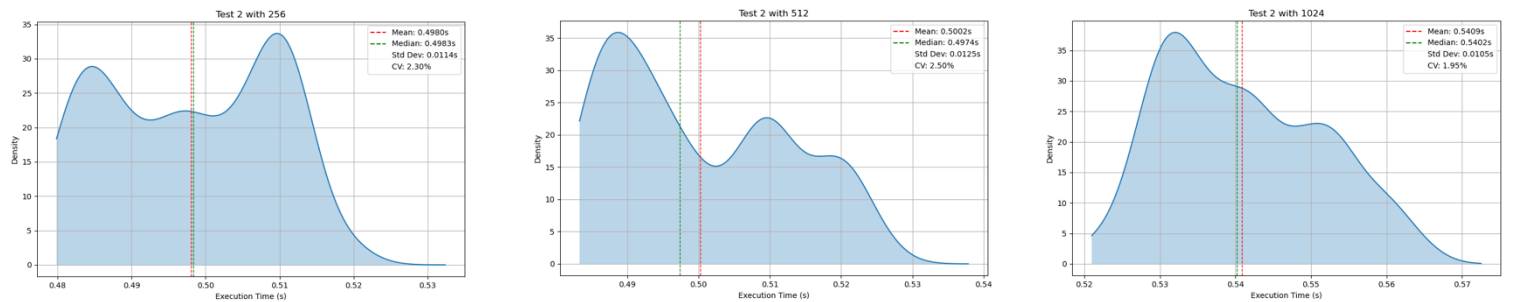
CUDA

I tempi misurati sono stati elaborati da grafici che presentano la loro mediana, media e distribuzione, in modo da osservare particolarità relative all'esecuzione.

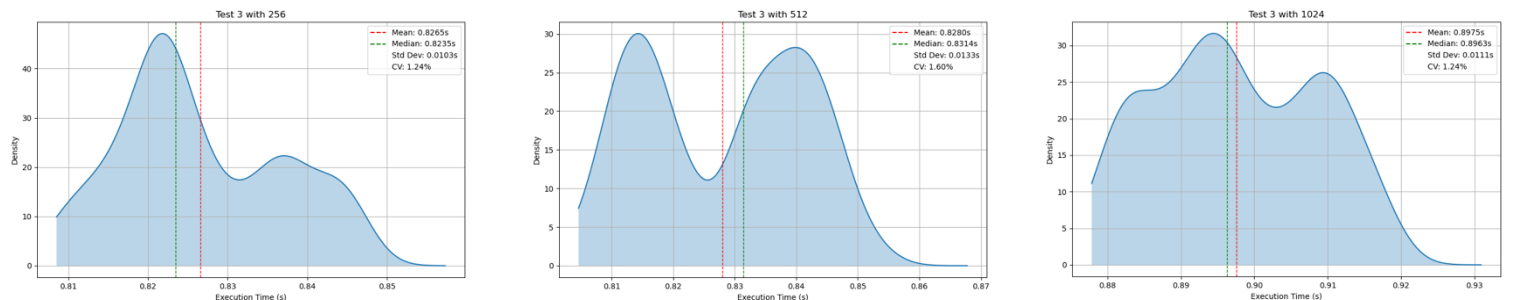
Primo test



Secondo test



Terzo test



All'osservazione dei grafici, salta subito all'occhio che le deviazioni standard hanno valori molto bassi in tutti i test, intorno al 0.011, quindi si potrebbe affermare che i dati sono stabili e consistenti. Questo può essere confermato anche dalla poca distanza tra la media e la mediana in tutti i casi. Tuttavia, osservando il coefficiente di variazione, si nota che all'aumentare della dimensione dell'input la dispersione relativa dei tempi di esecuzione tende a diminuire (dal 7.5% all'1.2%). Questo suggerisce, al contrario della versione MPI+OMP, una maggiore stabilità del tempo di esecuzione per input più grandi, probabilmente dovuta al fatto che con input piccoli, il rumore (overhead, scheduling, ecc.) incide di più.

Il grafico sottostante è invece inerente allo speed up. Dall'analisi dei risultati emerge l'alta affidabilità di 256 threads, mentre il caso con 512 threads non riesce a offrire performance migliori, sottolineando quindi che in questo caso l'aumento del numero di risorse non comporta un incremento delle prestazioni. Inoltre, lo speedup del caso di 1024 threads tende a divergere, per

quanto comunque crescente, specialmente nei test di dimensione maggiore, a causa di un probabile aumento dell'overhead di parallelizzazione. Il miglior compromesso tra parallelismo ed efficienza si ottiene, quindi, con 256 threads.

